Administration système Linux

Informatique — Licence 2 (L2)

Cours INF-L2-09

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours

Passer de l'utilisateur à l'administrateur : installer une distribution Linux, gérer les comptes, les groupes et les permissions, installer des logiciels, piloter les services systemd, automatiser avec des scripts et cron, administrer à distance par SSH et superviser la machine grâce aux journaux système.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Installer et découvrir une distribution Linux
• Leçon 2 — Utilisateurs, groupes et permissions sur les fichiers
• Leçon 3 — Gestion des paquets et installation de logiciels
• Leçon 4 — Gestion des processus et des services avec systemd
• Leçon 5 — Automatisation : scripts shell et tâches planifiées avec cron
• Leçon 6 — Partage de fichiers et accès distant par SSH
• Leçon 7 — Supervision de base et lecture des journaux système
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En première année, tu as appris à dialoguer avec la machine : naviguer dans les dossiers, créer des fichiers, enchaîner les commandes. Ce cours franchit un cap : tu ne seras plus seulement un utilisateur de Linux, tu deviendras son administrateur. Administrer, c'est décider qui peut se connecter, qui a le droit de lire ou de modifier chaque fichier, quels programmes tournent en permanence, et comment la machine réagit quand quelque chose casse. C'est le quotidien des techniciens système, des développeurs qui déploient leurs applications et de tous ceux qui entretiennent des serveurs, y compris ici à Bamako où de plus en plus d'entreprises et d'administrations s'appuient sur des serveurs Linux sobres, stables et économiques en licences. Ce rôle d'administrateur est aussi celui qui contrôle les accès : une erreur de sa part peut ouvrir une brèche, une bonne pratique de sa part protège toute l'organisation.

Tout le cours se déroule sur une même machine fil rouge : un petit serveur Ubuntu nommé srv-iam, administré par le compte adminiam. Nous créerons des comptes pour les étudiants, nous protégerons un dossier partagé avec des permissions calculées au plus juste, nous installerons des logiciels, nous piloterons des services, puis nous automatiserons la sauvegarde des documents chaque nuit à 2 heures du matin. Enfin, nous apprendrons à nous connecter à ce serveur à distance par SSH, comme le font tous les administrateurs professionnels, et à lire ses journaux pour comprendre ce qui s'y passe réellement. Chaque leçon se termine par un exercice corrigé ; les TD et le TP final reprennent l'ensemble de la démarche sur des situations complètes, de la mise en service d'un poste à la résolution d'une panne documentée.

La règle d'or de l'administration, à retenir dès maintenant : ne jamais taper une commande de modification sans savoir ce qu'elle fait, et toujours vérifier le résultat avec une commande de lecture. Un administrateur prudent teste sur une machine virtuelle, documente ses actions et ne donne jamais plus de droits que nécessaire. Ce principe du moindre privilège reviendra dans presque toutes les leçons : il protège la machine contre les erreurs des utilisateurs… et contre celles de l'administrateur lui-même. Installe une machine virtuelle Ubuntu dès le début du cours : chaque exemple doit être reproduit de tes propres mains, car l'administration système ne s'apprend pas en lisant, mais en pratiquant dans un environnement où l'on peut se tromper sans danger, recommencer et comparer ses résultats à ceux du cours.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Installer une distribution Linux (machine virtuelle ou poste réel) et identifier sa version, son noyau et ses ressources.
- Créer, modifier et supprimer des utilisateurs et des groupes avec useradd, usermod, passwd et groupadd.
- Lire et modifier les permissions d'un fichier ou d'un dossier, en notation symbolique et en octal (chmod, chown).
- Installer, mettre à jour et supprimer des logiciels avec un gestionnaire de paquets (apt).
- Démarrer, arrêter, activer et diagnostiquer des services avec systemctl et systemd.
- Écrire un script shell simple et planifier son exécution automatique avec cron.
- Se connecter à une machine distante par SSH, transférer des fichiers avec scp et lire les journaux système pour diagnostiquer un incident.

Leçon 1 — Installer et découvrir une distribution Linux

Une distribution Linux est un ensemble complet et cohérent : le noyau Linux, les outils de base, un gestionnaire de paquets, des logiciels et souvent un environnement graphique. Debian, Ubuntu (construite sur Debian), Fedora ou Arch Linux partagent le même noyau mais font des choix différents : stabilité éprouvée pour Debian et les serveurs, versions récentes pour Fedora, contrôle total pour Arch. Pour apprendre et pour administrer des serveurs, la famille Debian/Ubuntu est le choix le plus répandu : sa documentation francophone est immense et la commande apt y règne. Les versions LTS (Long Term Support) d'Ubuntu, publiées tous les deux ans, sont maintenues cinq ans : ce sont elles qu'on installe sur un serveur.

Il existe quatre façons d'installer Linux pour apprendre. La machine virtuelle (VirtualBox, par exemple) fait tourner Linux dans une fenêtre, sans toucher au système principal : c'est la méthode recommandée pour ce cours, car on peut casser et recommencer sans risque. Le démarrage double (dual boot) installe Linux à côté de Windows sur le même disque : performant, mais le partitionnement demande de la prudence. La clé USB « live » permet d'essayer sans rien installer. Enfin, sous

Windows, WSL offre un vrai terminal Linux intégré, suffisant pour les commandes mais limité pour l'administration des services. Dans tous les cas, l'installation suit les mêmes étapes : choisir la langue et le clavier, décider du partitionnement, créer le premier utilisateur — qui aura, sur Ubuntu, le droit d'utiliser sudo.

Juste après l'installation, l'administrateur fait connaissance avec sa machine. Où suis-je connecté (hostname), quelle distribution et quelle version (lsb_release -a ou le fichier /etc/os-release), quel noyau (uname -r), combien de mémoire et d'espace disque (free -h, df -h) ? Ces commandes d'identification sont le premier réflexe de tout dépannage : on ne répare pas ce qu'on n'a pas identifié. On en profite aussi pour appliquer immédiatement les mises à jour de sécurité et pour noter les informations dans un carnet d'administration. L'arborescence découverte en L1 prend alors tout son sens : /home pour les utilisateurs, /etc pour la configuration, /var pour les données qui varient (dont les journaux) et /usr pour les programmes.

L'essentiel
Choisir une distribution LTS pour la stabilité, s'entraîner en machine virtuelle, puis identifier sa machine dès la première connexion : distribution, noyau, mémoire, disques — avant toute autre action.

Exemple — Première connexion sur srv-iam : faire connaissance avec la machine

```
$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description: Ubuntu 24.04.1 LTS
Release: 24.04
Codename: noble

$ uname -r
6.8.0-45-generic

$ hostnamectl
Static hostname: srv-iam
Icon name: computer-vm
Operating System: Ubuntu 24.04.1 LTS
Kernel: Linux 6.8.0-45-generic
Architecture: x86-64

$ whoami
adminiam

$ pwd
/home/adminiam

$ df -h /
Filesystem Size Used Avail Use% Mounted on
/dev/sda2 40G 11G 27G 29% /

$ free -h
total used free shared buff/cache available
Mem: 7.7Gi 1.2Gi 5.1Gi 8.0Mi 1.4Gi 6.2Gi
Swap: 2.0Gi 0B 2.0Gi
```

À retenir

- Une distribution = noyau Linux + outils + gestionnaire de paquets + logiciels, avec une philosophie propre (stabilité pour Debian, fraîcheur pour Fedora).
- Pour apprendre : machine virtuelle d'abord — on y casse sans conséquence ; sur un serveur, choisir une version LTS maintenue plusieurs années.
- Premiers réflexes après installation : lsb_release -a, uname -r, free -h, df -h, puis les mises à jour de sécurité.

Exercice

Sur ta machine virtuelle Ubuntu, exécute les commandes d'identification vues dans la leçon et réponds à ces questions : quelle est la version exacte de ta distribution ? Quelle version du noyau tourne ? Combien reste-t-il d'espace libre sur le disque principal, en Go et en pourcentage ? Combien de mémoire totale la machine possède-t-elle ?

✔ Corrigé
On obtient les réponses une par une : lsb_release -a affiche la distribution et sa version (par exemple Ubuntu 24.04.1 LTS, nom de code noble). uname -r donne la version du noyau (par exemple 6.8.0-45-generic). df -h / affiche la ligne du système de fichiers racine : la colonne Avail donne l'espace libre lisible (par exemple 27G) et Use% le pourcentage utilisé (29 %, donc 71 % de libre). free -h donne la mémoire totale sur la ligne Mem (par exemple 7.7Gi). Ces quatre réponses forment la « carte d'identité » de la machine : un administrateur les note toujours avant d'intervenir, car un même symptôme (un site lent, par exemple) n'a pas la même cause sur une machine neuve et sur une machine dont le disque est plein à 98 %.

Leçon 2 — Utilisateurs, groupes et permissions sur les fichiers

Linux est né multi-utilisateur : plusieurs personnes partagent la même machine, et le système doit garantir que chacune ne touche qu'à ce qui la concerne. Au sommet trône root, le super-utilisateur (identifiant UID 0), qui peut tout faire — c'est pourquoi on ne travaille jamais connecté en root au quotidien : on utilise un compte ordinaire et la commande sudo pour élever temporairement ses droits. Les comptes ordinaires reçoivent un UID à partir de 1000. Toutes les informations vivent dans des fichiers texte : /etc/passwd décrit les comptes, /etc/shadow conserve les mots de passe chiffrés (lisible seulement par root), /etc/group liste les groupes. On gère les comptes avec useradd (créer), usermod (modifier, par exemple ajouter à un groupe), passwd (définir le mot de passe) et userdel (supprimer, avec -r pour emporter le dossier personnel).

Chaque fichier et chaque dossier appartient à un utilisateur propriétaire et à un groupe propriétaire. Les droits se lisent avec ls -l : trois groupes de trois lettres rwx. r (read, valeur 4) autorise la lecture, w (write, valeur 2) la modification, x (execute, valeur 1) l'exécution d'un programme — ou, pour un dossier, le droit de le traverser et d'accéder à son contenu. La première série concerne le propriétaire (user), la deuxième le groupe (group), la troisième tous les autres (others). Ainsi -rw-r--r-- signifie : fichier ordinaire, le propriétaire lit et écrit, le groupe et les autres lisent seulement. Sans le droit x sur un dossier, impossible d'y entrer même si l'on connaît le nom des fichiers qu'il contient.

On change les droits avec chmod, de deux façons équivalentes. La notation octale additionne les valeurs : pour donner lecture+écriture+exécution au propriétaire (4+2+1 = 7), lecture+exécution au groupe (4+1 = 5), rien aux autres (0), on écrit chmod 750. La notation symbolique modifie un droit précis : chmod g+w ajoute l'écriture au groupe, chmod o-r retire la lecture aux autres, chmod u+x rend un script exécutable par son propriétaire. chown change le propriétaire (sudo chown fatou fichier) et chgrp le groupe propriétaire. La discipline à retenir : donner le minimum utile. Le fameux chmod 777 (tout le monde peut tout faire) est un aveu d'échec et une faille de sécurité, jamais une solution.

L'essentiel
Trois acteurs (propriétaire, groupe, autres) × trois droits (r=4, w=2, x=1) : chmod 750 = rwxr-x---. Les groupes servent à partager proprement, sudo remplace le travail en root, et 777 est interdit.

Exemple — Créer le compte de Fatou et protéger un dossier partagé

```
$ sudo useradd -m -s /bin/bash fatou
$ sudo passwd fatou
New password: ********
Retype new password: ********
passwd: password updated successfully

$ id fatou
uid=1001(fatou) gid=1001(fatou) groups=1001(fatou)

$ sudo usermod -aG sudo fatou
$ id fatou
uid=1001(fatou) gid=1001(fatou) groups=1001(fatou),27(sudo)

$ ls -l /home/fatou/.bashrc
-rw-r--r-- 1 fatou fatou 3771 sept. 12 09:14 /home/fatou/.bashrc

$ mkdir ~/partage && touch ~/partage/notes.txt
$ ls -l ~/partage
-rw-r--r-- 1 adminiam adminiam 0 sept. 12 09:20 notes.txt

$ chmod 750 ~/partage
$ ls -ld ~/partage
drwxr-x--- 2 adminiam adminiam 4096 sept. 12 09:20 /home/adminiam/partage

$ chmod 640 ~/partage/notes.txt
$ chmod g+w ~/partage/notes.txt
$ ls -l ~/partage/notes.txt
-rw-rw---- 1 adminiam adminiam 0 sept. 12 09:21 /home/adminiam/partage/notes.txt
```

À retenir

- useradd -m crée le compte ET son dossier personnel ; usermod -aG ajoute à un groupe SANS retirer les autres groupes (oublier -a écrase tout).
- En octal : r = 4, w = 2, x = 1 ; on additionne par série. 644 = rw-r--r-- pour un fichier ordinaire, 750 = rwxr-x---.
- Sur un dossier, x signifie « traverser » : sans lui, aucun accès au contenu, même avec r sur les fichiers.

Exercice

1) Écris la commande complète pour créer l'utilisateur moussa avec son dossier personnel et le shell /bin/bash, puis pour lui définir un mot de passe. 2) Que signifient exactement les permissions -rwxr-x--- sur un script ? Qui peut faire quoi ? 3) On veut qu'un fichier secret.txt ne soit lisible et modifiable QUE par son propriétaire, sans

aucun droit pour le groupe ni pour les autres : quelle commande chmod en octal faut-il appliquer ?

✔ Corrigé
1) sudo useradd -m -s /bin/bash moussa, puis sudo passwd moussa (le mot de passe est saisi deux fois, sans affichage). On vérifie avec id moussa : un UID ≥ 1000 et le groupe moussa doivent apparaître. 2) -rwxr-x--- : le premier caractère « - » indique un fichier ordinaire ; le propriétaire (rwx) peut lire, modifier et exécuter le script ; le groupe propriétaire (r-x) peut le lire et l'exécuter mais pas le modifier ; les autres (---) n'ont aucun droit, pas même la lecture. 3) Propriétaire : lecture + écriture = 4 + 2 = 6 ; groupe : 0 ; autres : 0. La commande est donc chmod 600 secret.txt, qui affiche ensuite -rw------- dans ls -l. C'est le réglage qu'on applique typiquement aux clés privées SSH de la leçon 6.

Leçon 3 — Gestion des paquets et installation de logiciels

Installer un logiciel à la main, fichier par fichier, serait ingérable : chaque programme dépend d'autres composants (ses dépendances) qu'il faut aussi installer, dans le bon ordre. Le gestionnaire de paquets résout ce problème. Un paquet est une archive qui contient un programme, sa configuration et la liste de ses dépendances. Les paquets sont publiés dans des dépôts (repositories), des serveurs officiels maintenus par la distribution. Sur Debian et Ubuntu, l'outil s'appelle APT et manipule des paquets .deb ; sur Fedora et RHEL, c'est DNF et le format .rpm. Le principe reste identique : le gestionnaire télécharge, vérifie l'intégrité, installe les dépendances et tient un registre de tout ce qui est installé, ce qui permet mises à jour et désinstallations propres.

Le geste quotidien de l'administrateur commence par sudo apt update : cette commande ne met à jour AUCUN logiciel, elle rafraîchit seulement la liste des paquets disponibles dans les dépôts — comme consulter un catalogue récent avant de commander. Ensuite, sudo apt upgrade installe les nouvelles versions des paquets déjà présents, et sudo apt install nom_du_paquet installe un logiciel et ses dépendances. Pour connaître l'existant : dpkg -l liste tous les paquets installés, apt show nom_du_paquet décrit un paquet. La désinstallation passe par apt remove (le programme part, sa configuration reste) ou apt purge (tout part), et apt autoremove nettoie les dépendances devenues orphelines.

D'autres voies existent, à connaître sans les confondre. dpkg -i fichier.deb installe un paquet téléchargé à la main, mais sans résoudre les dépendances : à réserver aux cas particuliers, en vérifiant la provenance du fichier. Les formats universels Snap (Ubuntu) et Flatpak emballent le programme avec ses dépendances : plus lourds, mais isolés du système. Compiler depuis le code source reste possible pour les cas experts. La règle de sécurité prime sur tout : n'installer que depuis les dépôts officiels ou des sources de confiance vérifiées. Un paquet s'installe avec les droits root ; un paquet piégé, c'est toute la machine qui est compromise.

L'essentiel
Le gestionnaire de paquets installe depuis des dépôts de confiance et gère les dépendances. Séquence type : apt update (le catalogue), apt install (le logiciel), apt upgrade (les versions), apt autoremove (le ménage).

Exemple — Mettre à jour le catalogue et installer le moniteur htop

```
$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [512 kB]
Fetched 2 431 kB in 3s (810 kB/s)
Reading package lists... Done
12 packages can be upgraded. Run 'apt list --upgradable' to see them.

$ apt show htop
Package: htop
Version: 3.3.0-4build1
Priority: optional
Section: admin
Description: interactive process viewer

$ sudo apt install -y htop
Reading package lists... Done
Building dependency tree... Done
The following NEW packages will be installed:
htop
0 upgraded, 1 newly installed, 0 to remove and 12 not upgraded.
Setting up htop (3.3.0-4build1) ...

$ dpkg -l | grep htop
ii htop 3.3.0-4build1 amd64 interactive process viewer

$ sudo apt upgrade -y
...
$ sudo apt autoremove -y
Reading package lists... Done
```

À retenir

- apt update ≠ apt upgrade : le premier rafraîchit la liste des paquets disponibles, le second installe réellement les nouvelles versions.
- apt remove garde les fichiers de configuration, apt purge les supprime aussi ; autoremove chasse les dépendances orphelines.
- Sécurité : uniquement des dépôts officiels ou vérifiés — un paquet s'installe en root, il a donc tous les droits sur la machine.

Exercice

1) Un collègue affirme : « J'ai lancé apt update, mon système est à jour. » Explique-lui son erreur en une phrase précise. 2) Écris la suite de commandes pour installer l'outil tree (qui affiche l'arborescence des dossiers), vérifier qu'il est bien installé, puis le

désinstaller complètement, configuration comprise, en nettoyant ensuite les dépendances inutiles.

✔ Corrigé
1) apt update ne fait que télécharger la liste des versions disponibles dans les dépôts : tant que apt upgrade n'a pas tourné, aucun logiciel n'a été mis à jour — c'est comme lire un catalogue sans passer commande. 2) La suite complète : sudo apt update (catalogue frais, si ce n'est pas déjà fait), puis sudo apt install -y tree. Vérification : dpkg -l | grep tree doit afficher une ligne commençant par « ii », ou plus simplement la commande tree /home doit afficher l'arborescence. Désinstallation complète : sudo apt purge -y tree, puis sudo apt autoremove -y pour retirer les dépendances que tree était le seul à utiliser. On revérifie avec dpkg -l | grep tree : plus aucune ligne « ii » ne doit apparaître (une ligne « rc » signalerait une configuration restante, que purge aurait dû emporter).

Leçon 4 — Gestion des processus et des services avec systemd

Tout programme en cours d'exécution est un processus, identifié par un numéro unique, le PID. ps aux dresse la liste complète des processus avec leur propriétaire, leur consommation et leur commande de lancement ; top (ou htop, installé à la leçon 3) affiche la même information en temps réel, triée par consommation. Pour arrêter un processus, on lui envoie un signal avec kill : le signal par défaut, TERM (15), demande poliment la fermeture et laisse au programme le temps de sauvegarder ; le signal KILL (9), kill -9, coupe brutalement sans recours — on ne l'utilise qu'en dernier ressort, quand le processus refuse de répondre. Un processus qui tourne en arrière-plan, sans utilisateur connecté, au service des autres, s'appelle un démon (daemon).

Sur les distributions modernes, les démons sont des services pilotés par systemd, le système d'initialisation qui démarre en tout premier (PID 1) et lance tous les autres composants. Chaque service est décrit par une unité (unit) portant un nom comme ssh.service ou cron.service. La commande systemctl en est le tableau de bord : systemctl status ssh montre l'état détaillé, start et stop démarrent ou arrêtent le service immédiatement, restart le relance (utile après un changement de configuration). La nuance capitale : enable et disable ne démarrent rien, ils décident seulement si le service se lancera automatiquement au prochain démarrage de la machine. La commande combinée systemctl enable --now ssh active le démarrage automatique ET démarre le service tout de suite.

Quand un service refuse de démarrer, on ne devine pas : on lit. systemctl status affiche les dernières lignes du journal et un état explicite — active (running) pour un service sain, inactive (dead) pour un service arrêté normalement, failed pour un service en échec. L'outil journalctl, détaillé à la leçon 7, donne le journal complet du service (journalctl -u ssh). La démarche professionnelle est donc toujours la même : status pour constater, journal pour comprendre, correction de la cause (souvent une erreur de configuration ou un port déjà occupé), puis restart et nouvelle vérification. is-active et is-enabled répondent par un mot, pratiques dans les scripts.

L'essentiel
Un service est un processus démon géré par systemd (PID 1). start/stop agissent tout de suite, enable/disable décident du démarrage automatique ; en cas d'échec : status, puis le journal avec journalctl -u.

Exemple — Inspecter les processus et piloter le service SSH

```
$ ps aux | grep -E "sshd|cron" | grep -v grep
root 645 0.0 0.0 6832 3120 ? Ss 08:59 0:00 /usr/sbin/cron -f
root 812 0.0 0.1 15432 8232 ? Ss 08:59 0:00 /usr/sbin/sshd -D

$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled)
Active: active (running) since sam. 2026-10-10 08:59:12 UTC; 2h 3min ago
Main PID: 812 (sshd)
Tasks: 1 (limit: 9488)
Memory: 8.1M

$ sudo systemctl restart ssh
$ systemctl is-active ssh
active
$ systemctl is-enabled ssh
enabled

$ sudo systemctl enable --now cron
$ sudo journalctl -u ssh -n 3 --no-pager
oct. 10 11:02:14 srv-iam sshd[812]: Server listening on 0.0.0.0 port 22.
oct. 10 11:02:14 srv-iam sshd[812]: Server listening on :: port 22.
oct. 10 11:02:15 srv-iam systemd[1]: Started ssh.service.
```

À retenir

- kill envoie TERM (15) par défaut — fermeture propre ; kill -9 (KILL) est le dernier recours, sans sauvegarde possible.
- enable ≠ start : un service « enabled » démarrera au prochain boot mais peut être arrêté maintenant ; « enable --now » fait les deux.
- Trois états à distinguer dans status : active (running), inactive (dead), failed — et le journal explique toujours le failed.

Exercice

Après avoir modifié la configuration du serveur web Apache (service apache2), un administrateur lance systemctl enable apache2 et s'étonne que le site réponde toujours avec l'ancienne configuration. 1) Explique son erreur. 2) Donne les commandes correctes pour appliquer la nouvelle configuration immédiatement ET conserver le démarrage automatique. 3) Le lendemain, le service affiche failed : quelle commande lancer en premier, et que chercher dans sa sortie ?

✔ Corrigé
1) enable ne touche pas au service en cours d'exécution : il crée seulement le lien qui lancera apache2 au prochain démarrage. Le processus en mémoire continue donc avec l'ancienne configuration. 2) Il faut sudo systemctl restart apache2 (arrêt puis démarrage : la nouvelle configuration est lue), ou reload quand le service le propose pour un simple rechargement. Le restart n'enlève pas l'activation : enable reste acquis, vérifiable avec systemctl is-enabled apache2 qui répond enabled. La commande unique systemctl enable --now apache2, tapée dès le départ, aurait tout fait. 3) Première commande : sudo systemctl status apache2 — elle affiche l'état failed, l'heure de l'échec et les dernières lignes du journal, souvent suffisantes (erreur de syntaxe dans un fichier de configuration, port 80 déjà occupé par un autre service). On complète avec sudo journalctl -u apache2 -n 50 pour le détail, on corrige la cause, puis restart et nouveau status : la boucle constater → comprendre → corriger → vérifier est la méthode de tout dépannage de service.

Leçon 5 — Automatisation : scripts shell et tâches planifiées avec cron

Un administrateur ne refait pas cent fois la même suite de commandes : il l'écrit une fois dans un script shell, un simple fichier texte contenant les commandes, que bash exécute dans l'ordre. La première ligne, le shebang #!/bin/bash, indique l'interpréteur à utiliser. Le fichier doit ensuite être rendu exécutable (chmod +x, leçon 2) et lancé par son chemin, par exemple ./sauvegarde.sh ou /home/adminiam/sauvegarde.sh. Un script peut utiliser des variables (DATE=$(date +%F) capture la date du jour), recevoir des arguments ($1, $2), tester avec if et répéter avec for. Même un script de cinq lignes apporte le bénéfice essentiel : la tâche est faite à l'identique, sans oubli, et elle est documentée par son propre texte.

Pour exécuter un script à heure fixe sans être présent, Linux offre cron, un service qui réveille les tâches planifiées dans des tables nommées crontab. Chaque utilisateur possède la sienne, éditée par crontab -e et listée par crontab -l. Une ligne de crontab compte cinq champs de temps suivis de la commande : minute (0-59), heure (0-23), jour du mois (1-31), mois (1-12), jour de semaine (0-7, 0 et 7 = dimanche). L'étoile * signifie « chaque ». Ainsi 0 2 * * * lance la commande tous les jours à 2 h 00 précises, 30 3 * * 1 le lundi à 3 h 30, et */15 * * * * toutes les quinze minutes. Lire et écrire ces lignes sans erreur est une compétence d'administrateur de base.

Deux pièges guettent le débutant dans cron. D'abord, l'environnement d'exécution est minimal : pas de répertoire courant garanti, un PATH réduit — on écrit donc toujours les chemins ABSOLUS des commandes et des fichiers (/home/adminiam/sauvegarde.sh, jamais ~/sauvegarde.sh tout seul dans certains contextes, ni un simple nom de script). Ensuite, la sortie du script disparaît si on ne la redirige pas : on ajoute >> /home/adminiam/sauvegarde.log 2>&1 pour conserver traces et erreurs. La méthode sûre : écrire le script, le tester à la main jusqu'à obtenir le résultat voulu, le planifier à une heure proche pour le voir tourner sous cron, vérifier le journal produit, puis seulement le mettre à son heure définitive.

L'essentiel
Un script bash = des commandes dans un fichier, shebang #!/bin/bash, rendu exécutable. cron exécute selon cinq champs (minute heure jour mois jour-semaine) ; dans cron, toujours des chemins absolus et une sortie redirigée vers un journal.

Exemple — Un script de sauvegarde nocturne planifié par cron

```
$ cat ~/sauvegarde.sh
#!/bin/bash
# Sauvegarde quotidienne du dossier Documents de adminiam
DATE=$(date +%F)
DEST=/home/adminiam/sauvegardes
mkdir -p "$DEST"
tar -czf "$DEST/docs-$DATE.tar.gz" /home/adminiam/Documents
echo "Sauvegarde du $DATE terminee" >> /home/adminiam/sauvegarde.log

$ chmod +x ~/sauvegarde.sh
$ ~/sauvegarde.sh
$ ls -lh ~/sauvegardes/
-rw-r--r-- 1 adminiam adminiam 2.3M oct. 11 02:00 docs-2026-10-11.tar.gz

$ crontab -e
# Sauvegarde tous les jours a 2 h 00 du matin
0 2 * * * /home/adminiam/sauvegarde.sh
# Rapport d'activite chaque lundi a 3 h 30
30 3 * * 1 /home/adminiam/rapport.sh >> /home/adminiam/rapport.log 2>&1

$ crontab -l
0 2 * * * /home/adminiam/sauvegarde.sh
30 3 * * 1 /home/adminiam/rapport.sh >> /home/adminiam/rapport.log 2>&1
```

À retenir

- Les cinq champs cron, dans l'ordre : minute, heure, jour du mois, mois, jour de semaine (0 et 7 = dimanche).
- */15 dans le champ minute = toutes les 15 minutes ; une plage comme 8-18 dans le champ heure = de 8 h à 18 h.
- Dans cron : chemins absolus partout, et redirection de la sortie (>> fichier.log 2>&1) sinon aucune trace en cas d'erreur.

Exercice

1) Écris le script ~/etat.sh qui ajoute à la fin du fichier ~/etat.log une ligne contenant la date et l'heure (commande date) suivie de la charge de la machine (commande uptime). N'oublie ni le shebang ni la permission d'exécution. 2) Écris la ligne de crontab qui exécute ce script tous les jours à 6 h 15. 3) Que signifie la ligne 0 23 * * 0 /home/adminiam/menage.sh ?

✔ Corrigé
1) Le script : première ligne #!/bin/bash, puis la commande { date; uptime; } >> /home/adminiam/etat.log regroupée, ou deux lignes echo avec $(date) et $(uptime), chacune terminée par >> /home/adminiam/etat.log. Puis chmod +x ~/etat.sh, et un test manuel ~/etat.sh suivi de cat ~/etat.log pour vérifier qu'une ligne complète est apparue. 2) La ligne de planification : 15 6 * * * /home/adminiam/etat.sh — minute 15, heure 6, tous les jours du mois, tous les mois, tous les jours de semaine. 3) 0 23 * * 0 : minute 0, heure 23, jour de semaine 0, c'est-à-dire chaque dimanche à 23 h 00. Retenir que le dernier champ parle en jours de semaine : 0 et 7 désignent tous les deux dimanche, 1 = lundi, …, 6 = samedi.

Leçon 6 — Partage de fichiers et accès distant par SSH

Administrer un serveur ne signifie pas rester assis devant son écran : le serveur est souvent dans une autre pièce, un autre bâtiment, parfois un autre pays, et il n'a fréquemment ni écran ni clavier. L'accès distant par SSH (Secure Shell) est la réponse : un protocole qui ouvre un terminal distant entièrement chiffré, sur le port 22. Tout ce qui transite — mot de passe, commandes, résultats — est illisible pour qui intercepterait le réseau, contrairement aux ancêtres Telnet ou FTP en clair. Côté serveur, on installe le paquet openssh-server et on active le service ssh ; côté client, la commande ssh utilisateur@adresse ouvre la session, par exemple ssh fatou@192.168.1.20. À la toute première connexion, le client affiche l'empreinte (fingerprint) de la clé du serveur et demande confirmation : c'est le mécanisme qui permettra ensuite de détecter un serveur imposteur.

Se connecter par mot de passe fonctionne, mais l'authentification par clés est à la fois plus sûre et plus confortable. Le principe : une paire de clés liées mathématiquement — la clé publique, que l'on dépose sur le serveur dans le fichier ~/.ssh/authorized_keys, et la clé privée, qui ne quitte JAMAIS le poste de l'administrateur et reste protégée par ses permissions (chmod 600, leçon 2). On génère la paire avec ssh-keygen -t ed25519, puis on installe la clé publique avec ssh-copy-id utilisateur@serveur. Aux connexions suivantes, plus de mot de passe à taper : le serveur vérifie que le client possède la clé privée correspondant à une clé publique autorisée. Sur un serveur exposé à Internet, on va jusqu'à interdire les mots de passe pour ne laisser que les clés.

SSH transporte aussi les fichiers. scp copie un fichier à travers la connexion chiffrée : scp rapport.pdf fatou@192.168.1.20:/home/fatou/ envoie, et le sens inverse s'écrit en échangeant source et destination ; l'option -r copie un dossier entier. sftp offre la même chose en session interactive, et rsync synchronise efficacement de gros dossiers en ne transférant que les différences. Pour le partage permanent d'un dossier entre machines, d'autres services prennent le relais : NFS, natif dans le monde Linux, et Samba, qui rend un dossier Linux visible depuis Windows comme un partage réseau classique. Mais au quotidien de l'administrateur, le trio ssh + scp + clés couvre l'immense majorité des besoins.

L'essentiel
SSH (port 22) = terminal distant chiffré. Les clés remplacent avantageusement les mots de passe : la publique va sur le serveur (authorized_keys), la privée ne quitte jamais ton poste. scp copie les fichiers dans le même tunnel sécurisé.

Exemple — Activer SSH sur le serveur, s'y connecter et transférer un fichier

```
# Sur le serveur srv-iam : installer et activer le service
$ sudo apt install -y openssh-server
$ sudo systemctl enable --now ssh

# Depuis le poste de Fatou : premiere connexion
fatou@poste-fatou:~$ ssh adminiam@192.168.1.20
The authenticity of host '192.168.1.20 (192.168.1.20)' can't be established.
ED25519 key fingerprint is SHA256:uNiVztksCsX4c+/ExempleEmpreinteServeur.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.20' (ED25519) to the list of known hosts.
adminiam@192.168.1.20's password: ********
adminiam@srv-iam:~$ hostname
srv-iam
adminiam@srv-iam:~$ exit
logout
Connection to 192.168.1.20 closed.

# Transfert d'un fichier par scp (envoi)
fatou@poste-fatou:~$ scp rapport.pdf adminiam@192.168.1.20:/home/adminiam/
rapport.pdf 100% 245KB 12.3MB/s 00:00

# Authentification par cle : generation et installation
fatou@poste-fatou:~$ ssh-keygen -t ed25519
fatou@poste-fatou:~$ ssh-copy-id adminiam@192.168.1.20
fatou@poste-fatou:~$ ssh adminiam@192.168.1.20
adminiam@srv-iam:~$
```

À retenir

- Serveur SSH = paquet openssh-server + service ssh actif ; connexion par ssh utilisateur@adresse, port 22 par défaut.
- La première connexion enregistre l'empreinte du serveur dans ~/.ssh/known_hosts : un changement d'empreinte inattendu est une alerte, pas un détail.
- Clé privée = secret absolu (permissions 600, jamais copiée ni envoyée) ; seule la clé publique voyage vers les serveurs.

Exercice

1) Écris la commande scp qui télécharge le fichier /home/adminiam/sauvegardes/docs-2026-10-11.tar.gz depuis srv-iam (adresse 192.168.1.20, compte adminiam) vers le dossier courant du poste de Fatou. 2) Le serveur SSH d'une machine écoute exceptionnellement sur le port 2222 : comment se connecte-t-on et comment copie-t-on avec scp dans ce cas ? 3) Après l'installation de sa clé, Fatou reçoit toujours « Permission denied (publickey) ». Cite deux causes

probables liées aux permissions.

✔ Corrigé
1) Le téléchargement inverse source et destination : scp adminiam@192.168.1.20:/home/adminiam/sauvegardes/docs-2026-10-11.tar.gz . — le point final désigne le dossier courant local. Une barre de progression confirme le transfert. 2) Pour ssh, l'option est -p : ssh -p 2222 adminiam@192.168.1.20. Pour scp, attention au piège classique : l'option est un P majuscule, scp -P 2222 fichier adminiam@192.168.1.20:/home/adminiam/. Confondre -p et -P est l'erreur la plus fréquente des débutants. 3) Causes probables : le dossier ~/.ssh du serveur est trop ouvert (il doit être en 700) ou le fichier authorized_keys est accessible en écriture au groupe ou aux autres (il doit être en 600) — SSH refuse alors de faire confiance aux clés, par sécurité. Autre cause côté client : la clé privée locale n'est pas en 600, et le client refuse de l'utiliser. La correction passe par chmod 700 ~/.ssh et chmod 600 ~/.ssh/authorized_keys sur le serveur, chiffres calculés exactement comme à la leçon 2.

Leçon 7 — Supervision de base et lecture des journaux système

Superviser, c'est répondre à tout moment à quatre questions : la machine est-elle chargée, a-t-elle assez de mémoire, ses disques ont-ils de la place, ses services vivent-ils ? Les indicateurs de base se consultent en quelques commandes. uptime donne la durée de fonctionnement et la charge moyenne (load average) sur 1, 5 et 15 minutes — à comparer au nombre de cœurs : une charge durablement supérieure au nombre de cœurs signale une saturation. top détaille en direct les processus gourmands, free -h la mémoire (c'est la colonne available qui compte vraiment, Linux utilisant la mémoire libre comme cache), df -h l'occupation des disques et du -sh dossier le poids d'un dossier précis, pour traquer ce qui remplit un disque.

Tout ce que fait la machine est consigné dans des journaux. Historiquement, ce sont des fichiers texte dans /var/log : syslog pour l'activité générale (Debian/Ubuntu), auth.log pour les authentifications — connexions réussies, échecs, usages de sudo —, auxquels s'ajoutent les journaux propres à chaque application. systemd a ajouté journald, une base centralisée que l'on interroge avec journalctl : sans option, tout le journal ; -u ssh limite à un service ; --since « 1 hour ago » ou --since yesterday filtrent par période ; -f suit en direct ; -p err ne garde que les erreurs et au-dessus. On lit un fichier journal avec les outils de L1 : cat, less, grep et surtout tail -f pour suivre les événements pendant qu'ils se produisent.

La méthode de diagnostic compte plus que les commandes : partir du symptôme, mesurer, lire, conclure. Un site qui ne répond plus ? systemctl status du service, puis son journal à l'heure du plantage. Un disque passé de 60 % à 100 % en une nuit ? df -h pour constater, du pour localiser le dossier coupable — très souvent un journal ou une sauvegarde qui grossit sans rotation. Des dizaines de « Failed password » dans auth.log depuis une adresse inconnue ? C'est une tentative d'intrusion par force brute : on vérifie qu'aucune n'a réussi (les lignes Accepted), on s'assure que seuls les comptes nécessaires existent, on impose les clés SSH. Les journaux transforment des

impressions (« ça ne marche plus ») en faits datés et vérifiables : c'est la matière première de tout administrateur.

L'essentiel
Superviser = mesurer (uptime, free, df, du) puis lire les journaux (/var/log et journalctl). auth.log raconte les connexions, journalctl -u raconte la vie d'un service : le diagnostic va du symptôme aux faits datés.

Exemple — Tour de contrôle du matin sur srv-iam

```
$ uptime
10:24:31 up 6 days, 1:27, 2 users, load average: 0.15, 0.10, 0.08

$ df -h / /home
Filesystem Size Used Avail Use% Mounted on
/dev/sda2 40G 31G 6.8G 83% /
/dev/sda3 80G 22G 54G 29% /home

$ du -sh /var/log
248M /var/log

$ free -h
Mem: 7.7Gi 3.1Gi 1.9Gi 12Mi 2.7Gi 4.3Gi

$ grep -c "Failed password" /var/log/auth.log
37

$ tail -n 3 /var/log/auth.log
oct. 11 10:02:11 srv-iam sshd[2214]: Failed password for invalid user admin from
203.0.113.45 port 51234 ssh2
oct. 11 10:02:14 srv-iam sshd[2214]: Connection closed by 203.0.113.45 port 51234
[preauth]
oct. 11 10:05:40 srv-iam sshd[2301]: Accepted password for fatou from 192.168.1.35
port 49822 ssh2

$ sudo journalctl -u ssh --since "1 hour ago" --no-pager | tail -n 3
oct. 11 10:05:40 srv-iam sshd[2301]: Accepted password for fatou from 192.168.1.35
port 49822 ssh2
```

À retenir

- Charge (load average) : à comparer au nombre de cœurs ; mémoire : regarder la colonne available de free -h, pas free.
- auth.log = qui s'est connecté, d'où, avec succès ou échec ; une rafale de Failed password depuis une IP inconnue = tentative d'intrusion.
- journalctl : -u service, --since pour la période, -p err pour les erreurs, -f pour suivre en direct — l'outil de diagnostic n° 1.

Exercice

Ce matin, les utilisateurs se plaignent : « le serveur est devenu très lent ». Donne, dans l'ordre, les quatre commandes de mesure que tu lances et ce que tu cherches dans chacune. Puis précise le fichier journal que tu consultes pour vérifier si quelqu'un

a tenté de forcer une connexion SSH pendant la nuit, et la commande grep exacte pour compter les échecs.

✔ Corrigé
D'abord uptime : une charge moyenne très supérieure au nombre de cœurs confirme une saturation CPU (ou une attente disque). Puis top (ou htop) pour identifier LE processus responsable, par exemple une sauvegarde ou un programme bloqué qui monopolise le processeur. Puis free -h : si la colonne available est quasi nulle et que la swap est utilisée, la lenteur vient d'un manque de mémoire — les programmes « rament » sur le disque. Enfin df -h : un disque à 100 % ralentit ou bloque tout ce qui écrit, journaux et applications compris. Ces quatre mesures prennent une minute et désignent presque toujours le coupable. Pour la sécurité de la nuit : le fichier est /var/log/auth.log, et le comptage des échecs SSH se fait par grep -c « Failed password » /var/log/auth.log (avec sudo, car le fichier est protégé). On complète par grep « Accepted » /var/log/auth.log pour vérifier qu'aucune tentative n'a abouti ; si un Accepted inconnu apparaît, l'incident devient un incident de sécurité à traiter immédiatement (changer les mots de passe, révoquer les clés, analyser la session).

Travaux dirigés (TD)

TD 1 — Mettre en service le poste d'une nouvelle employée

Une PME de Bamako installe un poste Ubuntu pour Awa, nouvelle employée du service Comptabilité. L'administrateur doit : 1) créer le groupe compta s'il n'existe pas et le compte d'Awa (dossier personnel, shell /bin/bash) ; 2) placer Awa dans le groupe compta, sans lui donner les droits d'administration sudo ; 3) créer le dossier partagé /home/partage-compta appartenant à root et au groupe compta, avec des permissions telles que seuls le propriétaire et le groupe puissent y entrer, lire et écrire, et que les autres n'aient aucun droit ; 4) créer dedans un fichier budget.txt accessible en lecture-écriture au propriétaire et au groupe uniquement. Donne toutes les commandes, dans l'ordre, puis la ligne ls -l attendue pour le dossier et pour le fichier.

✔ Corrigé du TD 1
Dans l'ordre : sudo groupadd compta (création du groupe) ; sudo useradd -m -s /bin/bash awa (compte avec dossier personnel) ; sudo passwd awa (mot de passe initial, qu'Awa changera à sa première connexion) ; sudo usermod -aG compta awa (ajout au groupe, avec -a pour ne pas écraser d'éventuels autres groupes ; on ne touche pas au groupe sudo, donc Awa n'a aucun droit d'administration). Vérification : id awa doit afficher groups=…(compta) et pas sudo. Pour le partage : sudo mkdir -p /home/partage-compta ; sudo chgrp compta /home/partage-compta. Les droits demandés : propriétaire rwx (7), groupe rwx (7), autres rien (0), soit sudo chmod 770 /home/partage-compta. La ligne attendue : drwxrwx--- 2 root compta … /home/partage-compta — le d initial pour dossier, rwx pour root, rwx pour le groupe compta, --- pour les autres. Pour le fichier : sudo touch /home/partage-compta/budget.txt ; sudo chgrp compta /home/partage-compta/budget.txt ; sudo chmod 660 /home/partage-compta/budget.txt (rw pour propriétaire 6, rw pour groupe 6, rien pour autres 0). Ligne attendue : -rw-rw---- 1 root compta … budget.txt. Note de méthode : sans le x sur le dossier (le 7 inclut x), Awa ne pourrait même pas traverser le dossier pour atteindre le fichier — c'est l'erreur la plus classique de ce TD.

TD 2 — Le site du club ne répond plus : diagnostic méthodique

Le site web du club informatique, hébergé sur srv-iam (service apache2), affiche une erreur de connexion depuis ce matin. Le serveur, lui, répond au ping et une session SSH fonctionne. De plus, un message signale que la sauvegarde de la nuit a échoué. Donne la procédure complète de diagnostic : les commandes dans l'ordre, ce que chacune doit révéler, et les deux causes racines les plus probables que les journaux permettront de confirmer ou d'écarter. Termine par les commandes de remise en service et de vérification.

✔ Corrigé du TD 2
Étape 1, l'état du service : sudo systemctl status apache2. Si l'état est failed ou inactive, le coupable est localisé. Étape 2, le journal du service : sudo journalctl -u apache2 --since today (ou -n 50) — on y cherche l'heure et la raison de l'arrêt : erreur de configuration, ou « No space left on device », qui relierait les deux incidents (le site ET la sauvegarde). Étape 3, la mesure des disques : df -h — si la racine / est à 100 %, Apache ne peut plus écrire ses journaux ni ses fichiers temporaires et la sauvegarde tar a échoué faute de place : une seule cause explique tout. Étape 4, localiser le remplissage : sudo du -sh /var/* | sort -h (ou du sur /home) révèle le dossier anormal, souvent /var/log sans rotation ou un dossier de sauvegardes anciennes. Remise en service : libérer l'espace (supprimer les anciennes sauvegardes identifiées, vider le journal concerné après l'avoir consulté), puis sudo systemctl restart apache2, vérification par systemctl is-active apache2 (réponse active) et un test réel de la page (curl http://localhost ou navigateur). Étape 5, prévention : programmer une alerte ou un nettoyage — par exemple une ligne cron hebdomadaire qui supprime les sauvegardes de plus de 30 jours, et vérifier que logrotate tourne. Les deux causes racines à trancher par les journaux étaient donc : disque plein (cause commune aux deux pannes) ou erreur de configuration récente d'Apache (qui n'expliquerait pas l'échec de la sauvegarde).

Travaux pratiques (TP)

TP 1 — Administrer ton serveur de TP de A à Z

Sur ta machine virtuelle Ubuntu (ou sur WSL avec services simulés quand c'est impossible), réalise la mise en service complète d'un mini-serveur d'équipe : 1) Identifie la machine (distribution, noyau, disque, mémoire) et consigne les résultats dans un fichier administration.md. 2) Crée un groupe etudiants, deux utilisateurs etu1 et etu2 membres de ce groupe, et un dossier /home/projet commun au groupe en 770. 3) Installe les paquets tree et htop à jour. 4) Écris un script ~/inventaire.sh qui écrit dans ~/inventaire.log la date, la liste des utilisateurs humains (fin de /etc/passwd) et l'espace disque libre. 5) Planifie ce script par cron chaque jour à 7 h 30. 6) Active le serveur SSH et connecte-toi depuis une deuxième machine virtuelle (ou depuis l'hôte) avec une clé, puis transfère administration.md par scp. 7) Termine par un tour de contrôle : charge, disque, et les trois dernières connexions dans auth.log.

✔ Attendus du TP 1
Attendus — 1) administration.md contient les sorties de lsb_release -a, uname -r, df -h / et free -h, datées. 2) id etu1 affiche le groupe etudiants ; ls -ld /home/projet affiche drwxrwx--- avec le groupe etudiants ; un test réel : etu2 peut y créer un fichier, un compte extérieur ne peut pas y entrer. 3) dpkg -l montre tree et htop en état ii, installés après un apt update. 4) Le script a un shebang #!/bin/bash, est en 750 ou 700 (chmod +x vérifié par ls -l), utilise des chemins absolus, et son exécution manuelle produit un inventaire.log complet et daté. 5) crontab -l affiche la ligne 30 7 * * * /home/<utilisateur>/inventaire.sh ; la planification a été validée une première fois à une heure de test proche avant d'être fixée à 7 h 30. 6) systemctl is-active ssh répond active ; la connexion par clé fonctionne sans mot de passe (ssh-copy-id utilisé, clé privée en 600) ; le fichier transféré est présent et identique des deux côtés (même taille avec ls -l). 7) Le tour de contrôle rapporte une charge faible, un disque sous les 85 %, et uniquement des Accepted password connus dans auth.log. Le TP est réussi si chaque étape est prouvée par une sortie de commande, pas seulement affirmée.

Exercices corrigés supplémentaires

Exercice 1

Traductions de permissions. 1) Donne la forme symbolique (lettres) des modes 754, 600 et 751. 2) Donne le mode octal correspondant à : propriétaire lecture-écriture-exécution, groupe lecture seule, autres aucun droit. 3) Même question pour : propriétaire lecture-écriture, groupe lecture, autres lecture.

✔ Corrigé
1) 754 : 7 = rwx, 5 = r-x, 4 = r--, soit rwxr-xr--. 600 : 6 = rw-, 0 = ---, 0 = ---, soit rw-------. 751 : 7 = rwx, 5 = r-x, 1 = --x, soit rwxr-x--x (les autres peuvent exécuter ou traverser, mais ni lire ni écrire : réglage parfois utilisé pour un dossier à traverser sans lister son contenu). 2) rwx = 7, r-- = 4, --- = 0 : mode 740. 3) rw- = 6, r-- = 4, r-- = 4 : mode 644 — le grand classique des fichiers ordinaires publics en lecture. Méthode infaillible : r vaut 4, w vaut 2, x vaut 1 ; on additionne par série de trois, propriétaire puis groupe puis autres, toujours dans cet ordre.

Exercice 2

Lecture et écriture de crontab. 1) Explique précisément quand s'exécute la ligne : */20 8-18 * * 1-5 /home/adminiam/controle.sh. 2) Écris la ligne qui lance /home/adminiam/nettoyage.sh chaque dimanche à 23 h 00. 3) Un étudiant écrit 25 * * * * en croyant planifier « le 25 de chaque mois » : que se passe-t-il réellement ?

✔ Corrigé
1) Champ par champ : minute */20 = toutes les 20 minutes (0, 20, 40) ; heure 8-18 = de 8 h à 18 h ; jour du mois * = tous ; mois * = tous ; jour de semaine 1-5 = du lundi au vendredi. Le script tourne donc toutes les 20 minutes, entre 8 h 00 et 18 h 40, les jours ouvrés uniquement. 2) Chaque dimanche à 23 h 00 : 0 23 * * 0 /home/adminiam/nettoyage.sh (le 7 à la place du 0 serait accepté aussi). 3) Piège classique : le premier champ est la MINUTE, pas le jour. Sa ligne lance la commande à la minute 25 de chaque heure, tous les jours — soit 24 exécutions par jour au lieu d'une par mois. Pour le 25 de chaque mois à, disons, 4 h 00, il fallait 0 4 25 * * : minute 0, heure 4, jour du mois 25. Retenir l'ordre des cinq champs comme une phrase : « Minute, Heure, Jour du mois, Mois, Jour de semaine ».

Exercice 3

mini-cas SSH. Depuis son poste, Sékou exécute : scp -p 2222 devoir.pdf adminiam@192.168.1.20:/home/adminiam/ et reçoit une erreur de connexion refusée, alors que le serveur SSH de la machine écoute bien sur le port 2222. 1) Explique l'erreur contenue dans sa commande. 2) Donne la commande correcte. 3) Pour la connexion ssh elle-même vers ce même port, quelle option utilise-t-on ?

✔ Corrigé
1) Pour scp, le port se précise avec un P MAJUSCULE (-P). Avec -p minuscule, scp comprend « preserve » (conserver les dates et permissions du fichier) et tente donc de se connecter sur le port par défaut 22, où rien n'écoute d'où la connexion refusée malgré un serveur SSH parfaitement fonctionnel sur 2222. 2) Commande correcte : scp -P 2222 devoir.pdf adminiam@192.168.1.20:/home/adminiam/. 3) Pour ssh, c'est l'inverse : l'option du port est un p minuscule, ssh -p 2222 adminiam@192.168.1.20. L'asymétrie entre les deux outils est historique et piégeuse ; le moyen mnémotechnique : scp copie le P de Port en capitale, comme pour se faire remarquer.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quel est l'identifiant (UID) de l'utilisateur root ?

   A) 1000

   B) 0

   C) 999

   D) 1

2. Que signifient les permissions 640 sur un fichier ?

   A) rw-r-----

   B) rwxr--r--

   C) rw-------

   D) r--r--r--

3. Quelle commande ne fait QUE rafraîchir la liste des paquets disponibles, sans rien installer ?

A) apt upgrade

B) apt install

C) apt update

D) apt autoremove

4. Que fait exactement systemctl enable ssh ?

A) Il démarre le service ssh immédiatement

B) Il active le démarrage automatique de ssh au boot, sans le démarrer maintenant

C) Il redémarre le service ssh

D) Il affiche le journal du service ssh

5. Quand s'exécute la ligne cron « 45 23 * * 0 commande » ?

A) Chaque dimanche à 23 h 45

B) Tous les jours à 23 h 45

C) Le jour 0 de chaque mois à 23 h 45

D) Toutes les 45 minutes

6. Sur quel port écoute un serveur SSH par défaut ?

A) 21

B) 443

C) 22

D) 80

7. Quelle commande affiche les journaux du seul service apache2 ?

A) ls /var/log/apache2 -u

B) journalctl -u apache2

C) systemctl log apache2

D) top -u apache2

8. Dans usermod -aG sudo fatou, que risque-t-on si l'on oublie l'option -a ?

A) Rien, -a est facultatif

B) Le mot de passe de fatou est effacé

C) fatou est retirée de tous ses autres groupes et ne garde que sudo

D) La commande échoue

9. Quelle commande copie le dossier photos/ entier vers un serveur distant ?

A) scp photos adminiam@192.168.1.20:/home/adminiam/

B) scp -r photos adminiam@192.168.1.20:/home/adminiam/

C) ssh -r photos adminiam@192.168.1.20

D) cp -r photos ssh://192.168.1.20

10. Dans quel fichier les mots de passe des utilisateurs sont-ils conservés, sous forme chiffrée ?

A) /etc/shadow

B) /etc/passwd

C) /etc/group

D) /var/log/auth.log

Corrigés du QCM

1. Réponse B — root a toujours l'UID 0 ; les comptes humains commencent en général à 1000.

2. Réponse A — 6 = rw- pour le propriétaire, 4 = r-- pour le groupe, 0 = --- pour les autres : rw-r-----.

3. Réponse C — apt update télécharge le catalogue des versions disponibles ; l'installation réelle passe par apt install ou apt upgrade.

4. Réponse B — enable règle le démarrage automatique ; pour démarrer en plus tout de suite : systemctl enable --now ssh.

5. Réponse A — Minute 45, heure 23, jour de semaine 0 = dimanche : rendez-vous hebdomadaire, le dimanche à 23 h 45.

6. Réponse C — SSH écoute sur le port TCP 22 ; 21 = FTP, 443 = HTTPS, 80 = HTTP.

7. Réponse B — journalctl -u unité filtre le journal systemd sur un service ; ici, celui d'Apache.

8. Réponse C — Sans -a (append), -G remplace la liste complète des groupes secondaires : fatou perdrait ses autres appartenances.

9. Réponse B — scp copie les fichiers ; pour un dossier entier, l'option -r (récursif) est indispensable.

10. Réponse A — /etc/shadow, lisible seulement par root ; /etc/passwd ne contient plus que les informations non sensibles des comptes.

Résumé — les notions importantes

• Identifier avant d'agir : lsb_release -a, uname -r, free -h, df -h donnent la carte d'identité de toute machine Linux ; pour apprendre, travailler en machine virtuelle sur une distribution LTS.

• Comptes et groupes vivent dans /etc/passwd, /etc/shadow et /etc/group ; useradd, usermod -aG et passwd les gèrent, et sudo remplace le travail permanent en root.

• Permissions : trois séries (propriétaire, groupe, autres) de r=4, w=2, x=1 ; chmod 750 = rwxr-x---, 644 = rw-r--r--, 600 = rw------- ; jamais de 777.

• Paquets : apt update rafraîchit le catalogue, apt install et apt upgrade installent, apt purge et autoremove nettoient ; uniquement des dépôts de confiance.

• systemd pilote les services : start/stop agissent immédiatement, enable/disable règlent le démarrage automatique ; en cas d'échec, status puis journalctl -u.

• Automatisation : un script bash (shebang, chmod +x) exécuté par cron selon cinq champs (minute, heure, jour du mois, mois, jour de semaine), avec chemins absolus et sortie redirigée.

• SSH sécurise l'administration à distance (port 22) : clés plutôt que mots de passe, clé privée secrète en 600, scp pour les fichiers, et auth.log + journalctl pour superviser et diagnostiquer.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 2 — Ligne de commandes Linux - partie 4 - Gestion des utilisateurs et des groupes

Chaîne : Ordirepar · Langue : Français · Niveau : Débutant

Gestion complète des utilisateurs et des groupes en ligne de commande : /etc/passwd, useradd, usermod, passwd, userdel, puis les droits d'accès avec chmod en octal. C'est exactement le programme de la leçon 2, démontré pas à pas dans un terminal.

https://www.youtube.com/watch?v=Kk8iC9xtvpM

Leçon 2 — 1-Apprendre chmod & chown — 3 exercices Linux pour débutants

Chaîne : Wael Tech School · Langue : Français · Niveau : Débutant

Trois exercices pratiques et courts sur les permissions : ajouter le droit d'exécution au propriétaire, retirer tous les droits aux autres, puis changer le propriétaire et le groupe d'un fichier avec chown. Parfait pour s'entraîner juste après la leçon 2.

https://www.youtube.com/watch?v=ql2SrJf-z3c

Leçon 5 — Introduction Script Bash sur Linux

Chaîne : IT-Connect - Florian · Langue : Français · Niveau : Débutant à intermédiaire

Créer son premier script Bash sur Linux : principe de l'interpréteur, écriture du script, exécution et enchaînements de commandes. La vidéo prolonge la leçon 5 et prépare l'automatisation des tâches d'administration, cron compris.

https://www.youtube.com/watch?v=gBCGeLjYBgQ

Leçon 6 — SSH - L'outil indispensable que tout utilisateur Linux devrait connaître

Chaîne : Ordirepar · Langue : Français · Niveau : Débutant

SSH de A à Z : installer le serveur openssh, s'y connecter depuis Linux, Windows et même un smartphone Android, et transférer des fichiers. La démonstration complète de la leçon 6, à suivre avec ta propre machine virtuelle.

https://www.youtube.com/watch?v=cNIn5jjtvO0

Leçon 6 — SSH : comprendre l'essentiel en 3 minutes

Chaîne : Cookie connecté · Langue : Français · Niveau : Débutant

L'essentiel de SSH en trois minutes : à quoi sert le protocole, les trois piliers de sa sécurité, le chiffrement et les cas d'usage. Idéal pour réviser la théorie de la leçon 6 avant un contrôle ou un TP.

https://www.youtube.com/watch?v=gn3vLuxl-Y8

Sources et références

• Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.

• Documentation Ubuntu francophone (doc.ubuntu-fr.org) — wiki communautaire : utilisateurs, permissions, apt, SSH et systemd y sont expliqués en français avec des exemples.

• « Debian Administrator's Handbook » (debian-handbook.info) — ouvrage de référence complet sur l'administration Debian, disponible en français et en accès libre.

• Les pages de manuel intégrées à Linux : man useradd, man chmod, man systemctl, man crontab — la source la plus fiable, déjà installée sur ta machine.

• Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !